



**Department of Computer Science
Air University Aerospace and Aviation Campus,
Kamra**

**Use Case & Abuse Case Diagrams
for
JPass: Java Password Manager**

Version 1.0

Prepared by:

Samreen Shoukat (245077)
Haleema Abid (245053)
Hamna Saleem (245047)

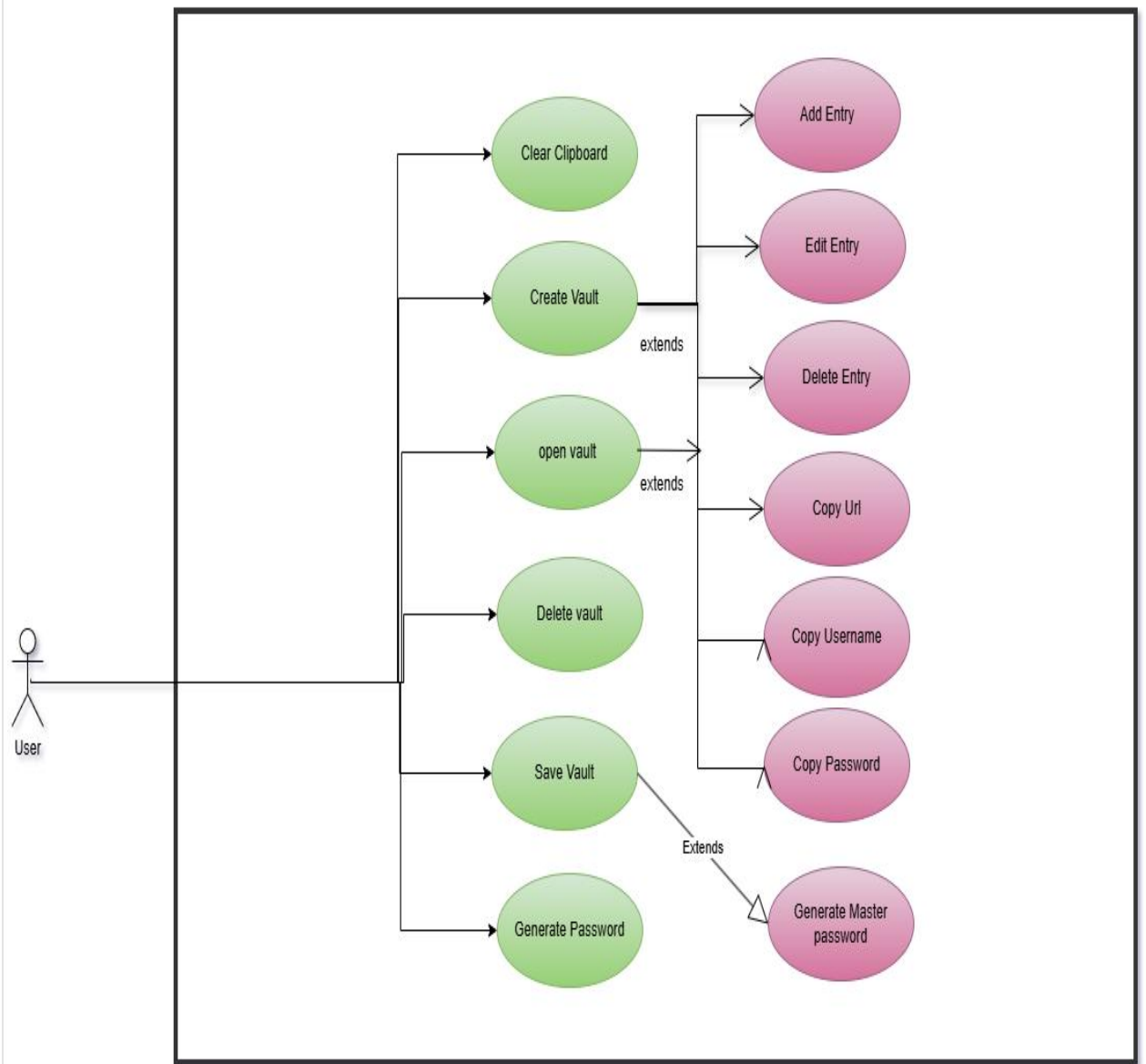
Submitted to:

Sir Zakauallah

March 4, 2026

Use Case Diagram

JPASS Universal Password Manager



1 Introduction

1.1 Purpose

JPass is a password manager that works on your computer and is built using Java. It helps you store all your passwords safely in one place. Each password is protected using AES-256 encryption, which keeps your information private and secure. With JPass, you only need to remember one master password to access all your accounts. It makes managing multiple passwords much easier. You can quickly add, edit, or delete your login details. The system also reduces the risk of using weak or repeated passwords. Overall, JPass simplifies password management while keeping your credentials safe. It is a useful tool for anyone who wants to stay secure online.

2 Overall Description

2.1 Product Perspective

JPass is a modern, standalone application designed to replace manual password storage. It keeps all passwords in encrypted local vaults for strong security. Users no longer need to remember or write down multiple passwords. The system makes accessing and managing credentials simple and safe. In the future, JPass may offer optional cloud integration. This will allow users to access their passwords securely from multiple devices. It is built to be easy to use while keeping data private and protected at all times.

2.2 Product Functions

The system provides the following features:

1. Store credentials securely in an encrypted vault.
2. Update or delete existing credentials.

3. Generate strong, random passwords automatically.
4. Search for specific entries within the password vault.
5. Export encrypted backups of stored credentials.
6. Copy passwords safely to the clipboard for quick use.

3 System Features

3.1 Master Password Authentication

3.1.1 Description and Priority

High priority. Ensures only authorized access.

3.1.2 Stimulus/Response Sequences

User enters master password → system validates → access granted or denied. System locks vault after multiple failed attempts.

3.1.3 Functional Requirements

REQ-1: System must allow creation of a master password. REQ-2:

System must deny access after 5 failed attempts.

REQ-3: System must never store master password in plaintext.

3.2 Password Storage

3.2.1 Description and Priority

High priority. Secure storage of all credentials.

3.2.2 Stimulus/Response Sequences

User adds/updates/deletes credentials → system encrypts → data saved in vault file.

3.2.3 Functional Requirements

REQ-4: System must encrypt all credentials using AES-256. REQ-5:

Users must be able to add, update, delete credentials.

REQ-6: System must maintain integrity of vault even if app closes unexpectedly.

3.3 Password Generation

3.3.1 Description and Priority

Medium priority. Helps users create strong passwords.

3.3.2 Stimulus/Response Sequences

User opens generator → selects length/complexity → system generates password → optionally saves to vault.

3.3.3 Functional Requirements

REQ-7: System must generate passwords with letters, numbers, symbols. REQ-8:

Users can configure length and complexity.

3.4 Password Retrieval and Search

3.4.1 Description and Priority

High priority. Efficiently retrieve credentials.

3.4.2 Stimulus/Response Sequences

User searches by username/URL → system returns matching credentials → allows copy to clipboard.

3.4.3 Functional Requirements

REQ-9: System must support search by username, account, or URL. REQ-10:
System must allow copying password to clipboard securely.

3.5 Vault Export/Backup

3.5.1 Description and Priority

Medium priority. Ensures data recovery.

3.5.2 Stimulus/Response Sequences

User exports vault → system encrypts → writes to selected location.

3.5.3 Functional Requirements

REQ-11: System must export encrypted vault.

REQ-12: Exported vault must be importable back to JPass.

4 Use Cases Description

- **Use Case 1:**

Create Vault

Description: User creates a new vault → system initializes encrypted vault file → ready for storing credentials.

Actors: User, System

- **Use Case 2:**

Add Entry

Description: User adds a new credential (username, password, URL) → system encrypts and saves

in vault.

Actors: User, System

- **Use Case 3:**

Edit Entry

Description: User updates an existing credential → system encrypts changes → vault updated.

Actors: User, System

- **Use Case 4:**

Delete Entry

Description: User deletes a credential → system removes it from vault → maintains vault integrity.

Actors: User, System

- **Use Case 5:**

Save Vault

Description: User saves the current state of vault → system writes encrypted vault to storage → ensures recovery after crash.

Actors: User, System

- **Use Case 6:**

Generate Password

Description: User opens password generator → selects length/complexity → system creates strong password → optionally adds to vault.

Actors: User, System

- **Use Case 7:**

Delete Vault

Description: User deletes entire vault → system removes encrypted file → confirms deletion.

Actors: User, System

- **Use Case 8:**

Copy Credentials

Description: User copies username/password → system securely places data on clipboard → clears clipboard after timeout.

Actors: User, System

- **Use Case 9:**

Find Entry

Description: User searches by username/account/URL → system returns matching credentials → user can view or copy them.

Actors: User, System

5 Conclusion

JPass system ensures secure management of passwords by providing a master-protected vault, encrypted storage, and efficient tools for generating, retrieving, and managing credentials. It enhances user security while keeping operations simple and reliable.

Abuse Case Diagram:

